Kuku - Find the Promos (Problem 1)

Solution summary | **python -m promo_finder run** | **streamlit run app.py** | code, tests, sample show and raw outputs are submitted with this document

1. Problem choice and framing

I chose Problem 1 because the input pack is real, the output can be checked against the brief's three tests (a hook in the first 3 s, a cliffhanger ending, makes sense on its own) and the work repeats for every new show. **This is a rebuild.** An audit of my first version found: its ranked list came from 4 hand-picked episodes; its scorer ranked my own human picks 395th-3,020th of 4,308 windows; and 9 of 15 documented steps crashed from a clean clone, so it could not be re-run on new input. v2 replaces it with one command that ranks **all 50 episodes** and runs on any folder or zip.

2. What it delivers

134 promos across 50 episodes (15 marked **post first**, max 2 per episode). Each row gives: episode, start/end of every segment in play order, the hook line (first 3 s), the cliffhanger line, **the line withheld after the cut**, a one-line why, five sub-scores and flags. Outputs: CSV, JSON, Markdown, quality report. Run time on the pack: 3.6 s, no API cost.

3. Assumptions (the brief was silent)

- The screenplay timestamps are the timeline; there is **no video**, so acting, audio and framing are invisible and an editor must watch each cut.
- Cut-in is the first frame of a shot whose dialogue/action starts within 3 s; cut-out is the last frame of a shot after a finished line, action or sound. Never through a spoken line or a sentence the script continues.
- Episodes end on a cliffhanger by design: 41 of 49 next-episode continuity notes open with a continuation phrase and 11 describe the cliffhanger outright. So each episode's list always includes its own episode-ending cut (48 of 134 promos), ranked by score like any other.
- Delivery tags such as (Cold, Threatening) are the best emotion signal in the pack (100% of lines have them). v1 discarded them; v2 uses them.

4. Approach: what is AI, what is code, where a human stays

Stage	Done by	Why
Parse, timestamps, cut points, validation, 9 hard checks	Code	Must be exact and repeatable. An LLM never writes a timestamp or a quote
Score ~34,149 clean-cut windows (hook 25, cliffhanger 30, standalone 15, tension 20, pace 10)	Rules (free, offline, explainable)	Cheap first pass over every window; every sub-score and penalty is shown
Re-score a shortlist (~168 calls for the whole pack), return scores + one-line why	LLM, optional	Semantic judgement keywords cannot make. Provider/model are config (Anthropic or any OpenAI-compatible API); cached; budget-guarded; bad answers fall back to the heuristic score
Pick distinct promos, rank, flag 45 of 134 for a check	Code	Editor watches the post-first list and every flagged promo

5. Scale and cost

Heuristic judge: 3.6 s for 50 episodes (linear in episodes), zero API cost, so 1,000 episodes is about a minute on one CPU. LLM judge (dry-run estimate, characters/4): **168 calls, ~117k input and ~24k output tokens** for 50 episodes, so about 3360 calls per 1,000 episodes; cost = tokens x your provider's price (not quoted here). Answers are cached by prompt hash, so re-runs and interrupted runs are free. Human time is the real cost: at 5 minutes per reviewed promo, 15 post-first promos per 50 episodes is about 75 minutes.

6. How I know it works - and where it does not

Check	Result	How far to trust it
Hard checks on all 134 promos: length 20-60 s, cuts inside episode, no cut through a line, hook within 3 s, quotes exist in the script, distinct, ranked, unique ids	9 / 9 pass	Structural only; says nothing about taste
Agreement with the **show's own cliffhanger notes** (next episode's continuity note describes how this one ended; n = 10 episodes; hit = key words in last 15s)	Top-1: **1/10**. Top-3: **6/10**. Baselines: final window 6/10, random windows 2.0/10, v1 top-3 4/10	Tiny n. Top-3 equals the 'final window' baseline because the episode-ending cut is always offered. Top-3 was 3/10 before my fix (below), so it is now in-sample
My earlier human picks (5 windows, 4 episodes; hit = IoU >= 0.5 with any of top-3)	v2 **4/5**; random 2.9/5; v1 5/5	Weak: v1's pool contained these picks and 20-60 s windows overlap a 40-60 s pick by chance. v2 top-1 differs on EP01 and EP22
Author's rejection case EP46 (0-60 s, resolved ending)	v2 top-1 is 89.2-143.8s (IoU 0.00 with it)	One case
Read of the top 15 by Claude from text (not an editor)	11 solid, 4 borderline, 0 clear failures	AI-assisted; verify a sample yourself

Changes after seeing results (disclosed): the first run showed the scorer ignored the writers' cliffhanger placement and the generator could not reach the real episode end in 18 of 50 episodes. The generator was fixed, the episode-end bonus raised 7 to 12 and a visual-reveal cue added; then the weights were frozen. Top-1 stayed 1/10, so on this proxy the tool does **not** beat

'take the last window'. Its added value is clean cut points, explanations and mid-episode alternatives.

Where it fails (read from the outputs):

- **Loaded vs procedural question.** It rewards an unanswered question but cannot tell "What wedding?" (answer is a bombshell) from "You served beside the accused?" (answer: "Yes."). Two top-15 picks end that way. The withheld line is shown so a human or the LLM can judge.
- A false 'resolved' penalty (3 promos, e.g. EP35) and a few weak forced episode endings (EP47, EP08). Bottom of the list: EP07, EP39 have no strong promo.
- Top picks come mostly from episodes 24-45 and lean on backstory; no early-episode bias is applied. On EP01 the human preferred the visual wedding opening; the tool prefers a spoken confrontation.
- Visual cliffhangers are only guessed from camera/action text; with no video, hook strength is judged from words alone.

7. AI tools used, and where they were wrong

ChatGPT built v1 (parsing, keyword scorer, dashboard). **Claude** audited v1 and wrote the v2 rebuild, tests, app and this document. Where they got it wrong: (a) the ChatGPT-built keyword scorer ranked the author's own top human picks 395th-3,020th of 4,308, and v1's only validation (Spearman 0.447 on 5 points) did not expose it; (b) Claude's first parser contained a 'zip-slip guard' that did nothing; (c) its first stitched-promo ids collided with normal ones (caught by a unit test); (d) its first quote checker reported 20 false failures (0 real). (b)-(d) were caught by self-review and tests, not by trusting the output.

8. Not done, next steps

- **The LLM judge has not been run against a live model** (no API access when it was built). The request/retry/JSON/cache/budget plumbing is tested against a mock server for both provider formats (42 tests in total). Next: one live run, then compare its verdicts with an editor on 20+ labelled promos.
- Experimental two-segment promos (--stitch) are built and validity-tested but never beat a continuous cut on this pack, so none are in the outputs.
- Next: add video/audio signals (shot energy, audio peaks), ask an editor to label 20+ promos for a proper hit-rate, and widen cliffhanger evidence beyond 10 episodes.